MODULE 3

RELATIONAL MODEL

CONTENT

- Structure of relational Databases
- Relational Algebra
- Extended Relational Algebra Operations
- Modification of Database
- Views
- Relational Calculus

STRUCTURE OF RELATIONAL DATABASE

- Based on tables
- The structure portion resemble an array of structs in C
- Column headers →Field names
- Each row →values of struct in the array

An Example Relation

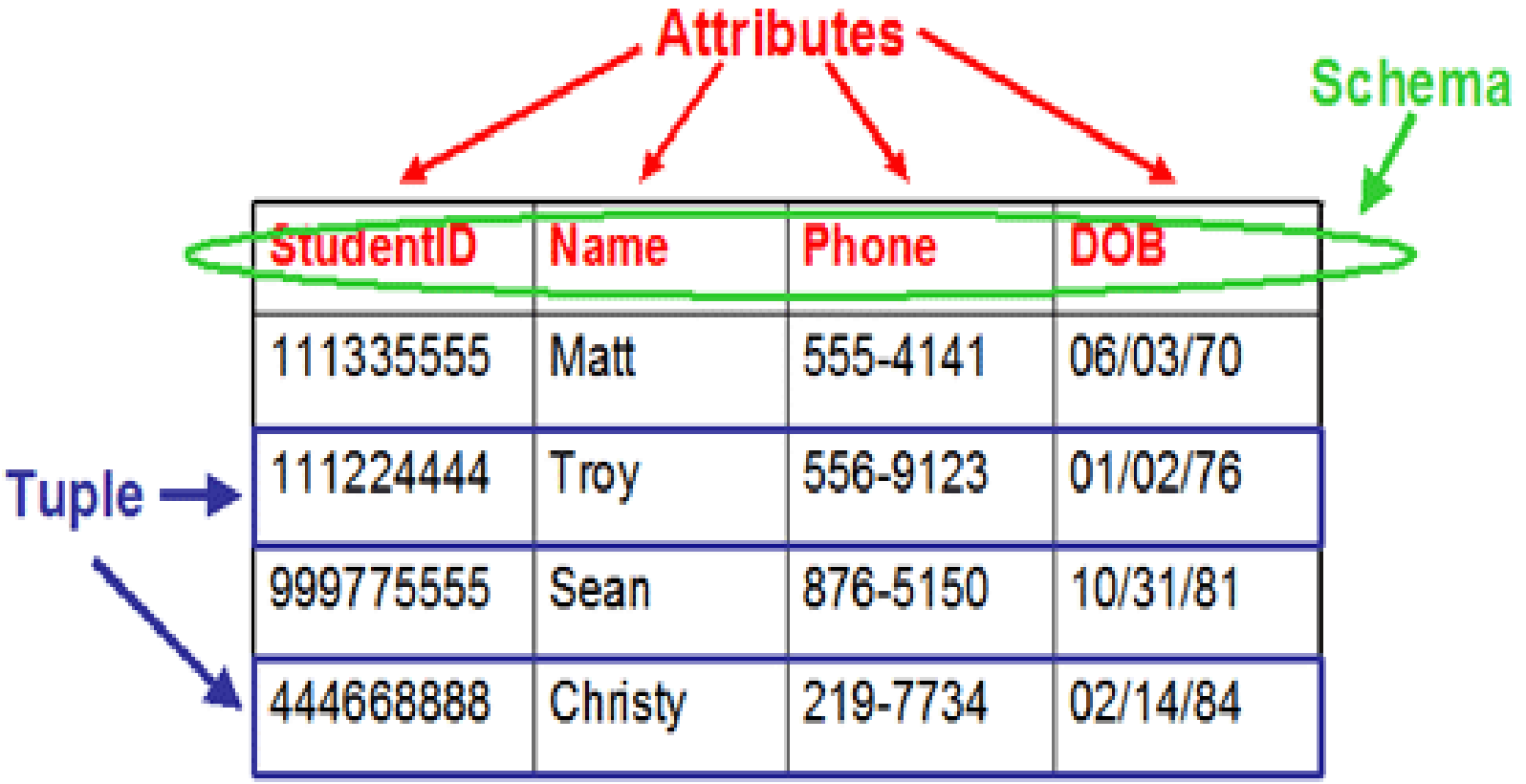
Title	Year	Length	Genre
Gone With the Wind	1939	231	Drama
Star Wars	1977	124	sciFi
Wayne's World	1992	95	comedy

Structure of Relational Database

- Relational model provides
  - A simple, limited approach to structuring data, yet is reasonably flexible so anything can be modeled
  - A limited, yet useful, collection of operations on data

Basics of the Relational Model

- Represent data as a two-dimensional table called a relation



Relational Data Model

- It is almost impossible to store a huge amount of data without proper manage.
- To manage and store data, many methods and models have been developed.
- Relational Database Model, which has proved to be the best data management model.

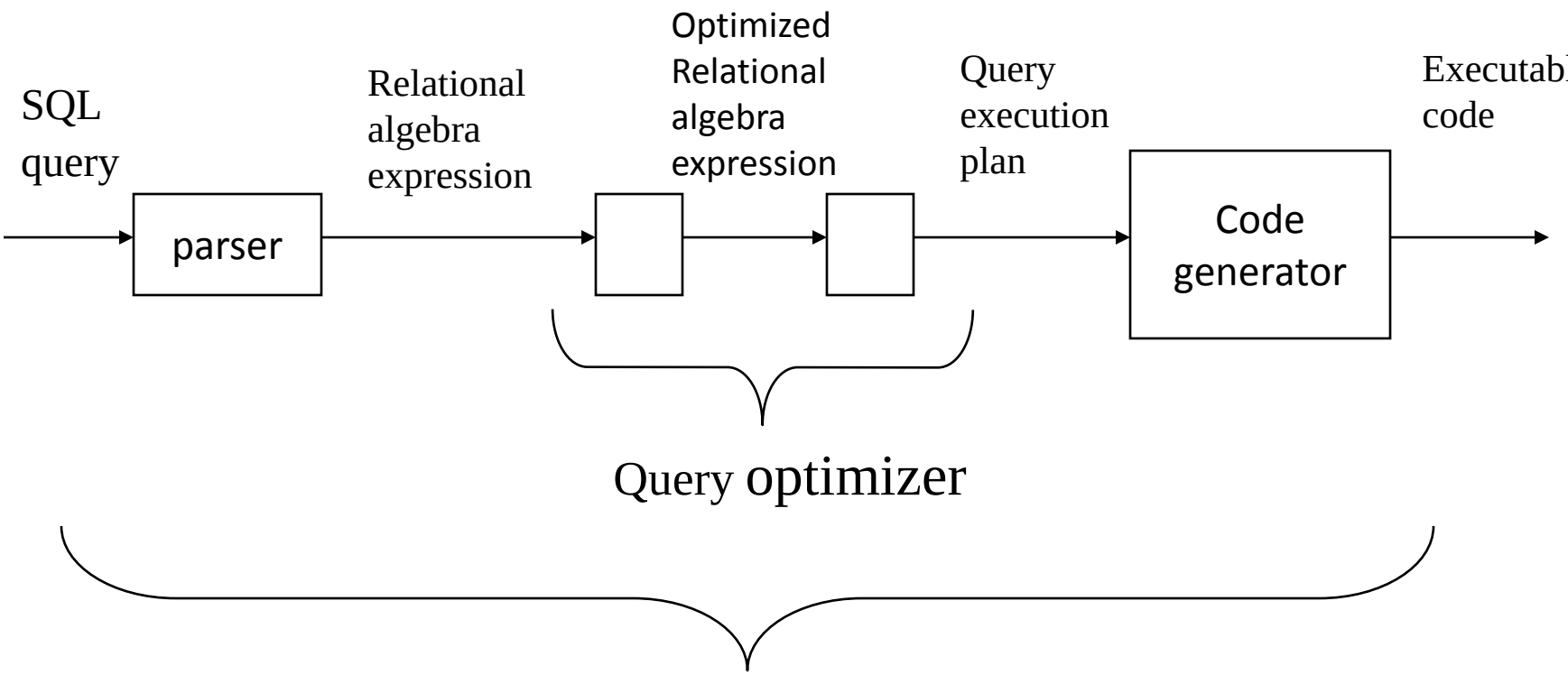
What is an Algebra?

- A language based on operators and a domain of values
- Operators map values taken from the domain into other domain values
- Hence, an expression involving operators and arguments produces a value in the domain
- When the domain is a set of all relations (and the operators are as described later), we get the *relational algebra*
- We refer to the expression as a *query* and the value produced as the *query result*

Relational Algebra

- *Domain*: set of relations
- *Basic operators*: select, project, union, set difference, Cartesian product
- *Derived operators*: set intersection, division, join
- *Procedural*: Relational expression specifies query by describing an algorithm (the sequence in which operators are applied) for determining the result of an expression

Relational Algebra in a DBMS



SYMBOLIC NOTATION

From the example, one can see that for complicated cases a large amount of the answer is formed from operator names, such as PROJECT and JOIN. It is therefore commonplace to use symbolic notation to represent the operators.

- SELECT ->σ (sigma)
- PROJECT -> π (pi)
- PRODUCT -> × (times)
- JOIN -> |x| (bow-tie)
- UNION -> ∪ (cup)
- INTERSECTION -> ∩ (cap)
- DIFFERENCE -> - (minus)
- RENAME ->ρ (rho)

SELECT OPERATOR

- Produce table containing subset of rows of argument table satisfying condition

$$\sigma_{condition}\, relation$$

- Example:

Person

Id	*Name*	*Address*	*Hobby*
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

$\sigma_{Hobby='stamps'}(Person)$

Id	*Name*	*Address*	*Hobby*
1123	John	123 Main	stamps
9876	Bart	5 Pine St	stamps

Selection Condition

- Operators: $<, \leq, \geq, >, =, \neq$
- Simple selection condition:
  - <*attribute*> *operator* <*constant*>
  - <*attribute*> *operator* <*attribute*>
- <*condition*> AND <*condition*>
- <*condition*> OR <*condition*>
- NOT <*condition*>

Selection condition - examples

- $\sigma_{Id>3000 \text{ Or } Hobby='hiking'}$ (Person)
- $\sigma_{Id>3000 \text{ AND } Id<3999}$ (Person)
- $\sigma_{\text{NOT}(Hobby='hiking')}$ (Person)
- $\sigma_{Hobby\neq'hiking'}$ (Person)

Project Operator

- Produces table containing subset of columns of argument table

$$\Pi_{attribute\ list}(relation)$$

- Example:

Person

Id	*Name*	*Address*	*Hobby*
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

$\Pi_{Name,Hobby}$(Person)

Name	*Hobby*
John	stamps
John	coins
Mary	hiking
Bart	stamps

Project Operator

- Example:

Person

Id	*Name*	*Address*	*Hobby*
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

$\Pi_{Name,Address}$(Person)

Name	*Address*
John	123 Main
Mary	7 Lake Dr
Bart	5 Pine St

Result is a table (no duplicates)

EXPRESSIONS

$$\Pi_{Id,\ Name}\ (\sigma_{Hobby='stamps'\ \text{OR}\ Hobby='coins'}\ (\text{Person})\ )$$

Id	*Name*	*Address*	*Hobby*
1123	John	123 Main	stamps
1123	John	123 Main	coins
5556	Mary	7 Lake Dr	hiking
9876	Bart	5 Pine St	stamps

Person

Id	*Name*
1123	John
9876	Bart

Result

SET OPERATORS

- Relation is a set of tuples => set operations should apply
- Result of combining two relations with a set operator is a relation => all its elements must be tuples having same structure
- Hence, scope of set operations limited to *union compatible relations*

Union Compatible Relations

- Two relations are *union compatible* if
  - Both have same number of columns
  - Names of attributes are the same in both
  - Attributes with the same name in both relations have the same domain
- Union compatible relations can be combined using *union*, *intersection*, and *set difference*

EXAMPLE

Tables:

Person (*SSN*, *Name*, *Address*, *Hobby*)

Professor (*Id*, *Name*, *Office*, *Phone*)

are not union compatible. However

Π_{Name} (Person) and Π_{Name} (Professor)

are union compatible and

Π_{Name} (Person) - Π_{Name} (Professor)

makes sense.

Cartesian Product

- If *R* and *S* are two relations, $R \times S$ is the set of all concatenated tuples <*x,y*>, where *x* is a tuple in *R* and *y* is a tuple in *S*
  - (*R* and *S* need not be union compatible)
- $R \times S$ is expensive to compute:
  - Factor of two in the size of each row
  - Quadratic in the number of rows

a	*b*
x1	x2
x3	x4

R

c	*d*
y1	y2
y3	y4

S

a	*b*	*c*	*d*
x1	x2	y1	y2
x1	x2	y3	y4
x3	x4	y1	y2
x3	x4	y3	y4

$R \times S$

EXAMPLE

Transcript (*StudId*, *CrsCode*, *Semester*, *Grade*)

Teaching (*ProfId*, *CrsCode*, *Semester*)

$\Pi_{StudId,\ CrsCode}$ (Transcript)[*StudId*, *CrsCode1*] $\times$
$\Pi_{ProfId,\ CrsCode}$(Teaching) [*ProfId*, *Crscode2*]

This is a relation with 4 attributes:
StudId, *CrsCode1*, *ProfId*, *CrsCode2*

Renaming

- Result of expression evaluation is a relation
- Attributes of relation must have distinct names. This is not guaranteed with Cartesian product
  - e.g., suppose in previous example *a* and *c* have the same name
- Renaming operator tidies this up. To assign the names $A_1, A_2, \ldots A_n$ to the attributes of the *n* column relation produced by expression *expr* use *expr* $[A_1, A_2, \ldots A_n]$

Renaming Example

•The rename operator returns an existing relation under a new name.

• $\sigma A(B)$ is the relation B with its name changed to A.

• For example, find the employees in the same Department as employee 3.

$$\pi_{\text{emp2.surname,emp2.forenames}} \Big(\sigma_{\text{employee.empno} = 3 \wedge \text{employee.depno} = \text{emp2.depno}} \big(\text{employee} \times (\rho_{\text{emp2}}\ \text{employee}) \big) \Big)$$

JOIN AND RENAMING

- **Problem**: *R* and *S* might have attributes with the same name – in which case the Cartesian product is not defined
- **Solution**:
  - Rename attributes prior to forming the product and use new names in *join-condition* .
  - Common attribute names are qualified with relation names in the result of the join

Theta Join – Example

Output the names of all employees that earn more than their managers.

$$\Pi_{\text{Employee}.Name}(\text{Employee} \bowtie_{MngrId=Id \text{ AND } Salary>Salary} \text{Manager})$$

The join yields a table with attributes:

Employee.*Name*, Employee.*Id*, Employee.*Salary*, *MngrId*
Manager.*Name*, Manager.*Id*, Manager.*Salary*

EQUIJOIN JOIN - EXAMPLE

Equijoin: Join condition is a conjunction of *equalities*.

$$\Pi_{Name,CrsCode}(\text{Student} \bowtie_{Id=StudId} \sigma_{Grade='A'}(\text{Transcript}))$$

Student

Id	*Name*	*Addr*	*Status*
111	John	…..	…..
222	Mary	…..	…..
333	Bill	…..	…..
444	Joe	…..	…..

Transcript

StudId	*CrsCode*	*Sem*	*Grade*
111	CSE305	S00	B
222	CSE306	S99	A
333	CSE304	F99	A

Mary	CSE306
Bill	CSE304

The equijoin is used very frequently since it combines related data in different relations.

NATURAL JOIN

- Special case of equijoin:
  - join condition equates *all* and *only* those attributes with the same name (condition doesn't have to be explicitly stated)
  - duplicate columns eliminated from the result

Transcript (*StudId*, *CrsCode*, *Sem*, *Grade*)
Teaching (*ProfId*, *CrsCode*, *Sem*)

$$\text{Transcript} \bowtie \text{Teaching} = \pi_{\textit{StudId, Transcript.CrsCode, Transcript.Sem, Grade, ProfId}} (\text{Transcript} \bowtie_{\textit{CrsCode}=\textit{CrsCode} \text{ AND } \textit{Sem}=\text{Sem}} \text{Teaching})$$

$$[\textit{StudId, CrsCode, Sem, Grade, ProfId}]$$

Natural Join (Con't)

- More generally:

$$R \bowtie S = \pi_{attr\text{-}list} (\sigma_{join\text{-}cond} (R \quad S))$$

where

attr-list = attributes (R) $\cup$ *attributes (S)*

(duplicates are eliminated) and *join-cond* has the form:

$A_1 = A_1$ AND … AND $A_n = A_n$

where

$\{A_1 \ldots A_n\}$ = *attributes(R)* $\cap$ *attributes(S)*

Natural Join Example

- List all Id's of students who took at least two different courses:

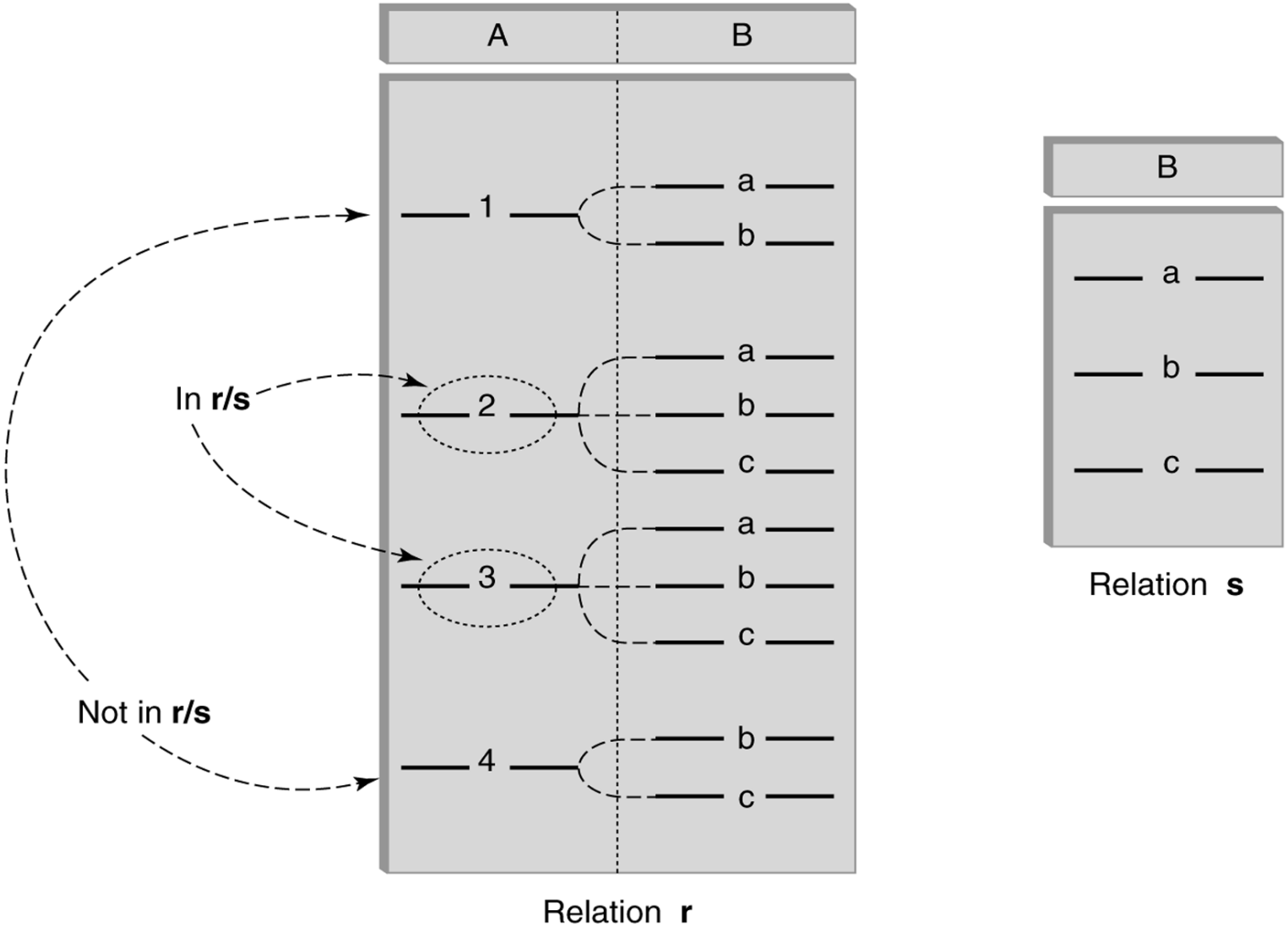
$$\Pi_{StudId}\,(\,\sigma_{CrsCode \neq CrsCode2}\,(\,\text{Transcript} \bowtie \text{Transcript}\,[StudId, CrsCode2, Sem2, Grade2]\,))$$

(don't join on *CrsCode*, *Sem*, and *Grade* attributes)

Division

- Goal: Produce the tuples in one relation, r, that match *all* tuples in another relation, s
  - $r\ (A_1, \ldots A_n, B_1, \ldots B_m)$
  - $s\ (B_1 \ldots B_m)$
  - *r/s*, with attributes $A_1, \ldots A_n$, is the set of all tuples <*a*> such that for every tuple <*b*> in *s*, <*a,b*> is in *r*
- Can be expressed in terms of projection, set difference, and cross-product

Division (contd.)



Division - Example

- List the Ids of students who have passed *all* courses that were taught in spring 2000
- Numerator: *StudId* and *CrsCode* for every course passed by every student
  - $\pi_{StudId,\ CrsCode}\ (\sigma_{Grade \neq \text{`F'}}\ (\text{Transcript})\ )$
- Denominator: *CrsCode* of all courses taught in spring 2000
  - $\Pi_{CrsCode}\ (\sigma_{Semester = \text{`S2000'}}(\text{Teaching})\ )$
- Result is *numerator/denominator*

Extended Relational-Algebra-Operations

- Generalized Projection
- Outer Join
- Aggregate Functions

Generalized Projection

- Extends the projection operation by allowing arithmetic functions to be used in the projection list.

$$\Pi_{F1, F2, \ldots, Fn}(E)$$

- E is any relational-algebra expression
- Each of F_1, F_2, …, F_n are are arithmetic expressions involving constants and attributes in the schema of E.
- Given relation *credit-info(customer-name, limit, credit-balance),* find how much more each person can spend:

$$\Pi_{customer\text{-}name,\ limit - credit\text{-}balance}\ (credit\text{-}info)$$

OUTER JOIN

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples form one relation that do not match tuples in the other relation to the result of the join.
- Uses *null* values:
  - *null* signifies that the value is unknown or does not exist
  - All comparisons involving *null* are (roughly speaking) **false** by definition.
    - Will study precise meaning of comparisons with nulls later

Outer Join – Example

- Relation *loan*

loan-number	branch-name	amount
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

- Relation borrower

customer-name	loan-number
Jones	L-170
Smith	L-230
Hayes	L-155

OUTER JOIN – EXAMPLE

- **Inner Join**

loan ⋈ *Borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith

- **Left Outer Join**

loan ⟕ Borrower

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	null

Outer Join – Example

- **Right Outer Join**

  loan ⟖ *borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	null	null	Hayes

- **Full Outer Join**

  loan ⟗ borrower

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	null
L-155	null	null	Hayes

Aggregate Functions and Operations

- **Aggregation function** takes a collection of values and returns a single value as a result.

  avg: average value
  min: minimum value
  max: maximum value
  sum: sum of values
  count: number of values

- **Aggregate operation** in relational algebra

$$_{G1, G2, \ldots, Gn}\mathcal{g}_{F1(A1), F2(A2), \ldots, Fn(An)}(E)$$

  - E is any relational-algebra expression
  - $G_1, G_2 \ldots, G_n$ is a list of attributes on which to group (can be empty)
  - Each F_i is an aggregate function
  - Each A_i is an attribute name

Aggregate Operation – Example

- Relation *r*:

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

$g_{\mathbf{sum(c)}}(r)$

sum-C
27

Aggregate Operation – Example

- Relation *account* grouped by *branch-name*:

branch-name	account-number	balance
Perryridge	A-102	400
Perryridge	A-201	900
Brighton	A-217	750
Brighton	A-215	750
Redwood	A-222	700

$$\text{branch-name}\ \mathcal{G}\ \text{sum(balance)}\ (\text{account})$$

branch-name	balance
Perryridge	1300
Brighton	1500
Redwood	700

Aggregate Functions (Contd.)

- Result of aggregation does not have a name
  - Can use rename operation to give it a name
  - For convenience, we permit renaming as part of aggregate operation

branch-name $\mathcal{G}$ **sum**(balance) **as** sum-balance (account)

Modification of the Database

- The content of the database may be modified using the following operations:
  - Deletion
  - Insertion
  - Updating
- All these operations are expressed using the assignment operator.

DELETION

- A delete request is expressed similarly to a query, except instead of displaying tuples to the user, the selected tuples are removed from the database.
- Can delete only whole tuples; cannot delete values on only particular attributes
- A deletion is expressed in relational algebra by:

$$r \leftarrow r - E$$

where r is a relation and E is a relational algebra query.

Deletion Examples

- Delete all account records in the Perryridge branch.

$$account \leftarrow account - \sigma_{branch\text{-}name = \text{“Perryridge”}}(account)$$

- Delete all loan records with amount in the range of 0 to 50

$$loan \leftarrow loan - \sigma_{amount \geq 0 \text{ and } amount \leq 50}(loan)$$

- Delete all accounts at branches located in Needham.

$$r_1 \leftarrow \sigma_{branch\text{-}city = \text{“Needham”}}(account \bowtie branch)$$

$$r_2 \leftarrow \sigma_{branch\text{-}name,\ account\text{-}number,\ balance}(r_1)$$

$$r_3 \leftarrow \sigma_{customer\text{-}name,\ account\text{-}number}(r_2 \bowtie depositor)$$

$$account \leftarrow account - r_2$$

$$depositor \leftarrow depositor - r_3$$

INSERTION

- To insert data into a relation, we either:
  - specify a tuple to be inserted
  - write a query whose result is a set of tuples to be inserted
- in relational algebra, an insertion is expressed by:

$$r \leftarrow r \cup E$$

  where r is a relation and E is a relational algebra expression.
- The insertion of a single tuple is expressed by letting E be a constant relation containing one tuple.

INSERTION EXAMPLES

- Insert information in the database specifying that Smith has $1200 in account A-973 at the Perryridge branch.

$$account \leftarrow account \cup \{(\text{“Perryridge”}, \text{A-973}, 1200)\}$$
$$depositor \leftarrow depositor \cup \{(\text{“Smith”}, \text{A-973})\}$$

- Provide as a gift for all loan customers in the Perryridge branch, a $200 savings account. Let the loan number serve as the account number for the new savings account.

$$r_1 \leftarrow (\sigma_{branch\text{-}name = \text{“Perryridge”}} (borrower \bowtie loan))$$
$$account \leftarrow account \cup \prod_{branch\text{-}name,\ account\text{-}number,\ 200} (r_1)$$
$$depositor \leftarrow depositor \cup \prod_{customer\text{-}name,\ loan\text{-}number} (r_1)$$

UPDATING

- A mechanism to change a value in a tuple without charging *all* values in the tuple
- Use the generalized projection operator to do this task

$$r \leftarrow \prod_{F1,\, F2,\, \ldots,\, Fl,} (r)$$

- Each F_i is either
  - the *i*th attribute of *r*, if the *i*th attribute is not updated, or,
  - if the attribute is to be updated F_i is an expression, involving only constants and the attributes of *r*, which gives the new value for the attribute

UPDATE EXAMPLES

- Make interest payments by increasing all balances by 5 percent.

$$account \leftarrow \Pi_{AN,\ BN,\ BAL * 1.05}(account)$$

where AN, BN and BAL stand for account-number, branch-name and balance, respectively.

- Pay all accounts with balances over $10,000 6 percent interest and pay all others 5 percent

$$account \leftarrow \Pi_{AN,\ BN,\ BAL * 1.06}(\sigma_{BAL > 10000}(account)) \cup \Pi_{AN,\ BN,\ BAL * 1.05}(\sigma_{BAL \leq 10000}(account))$$

Views

- In some cases, it is not desirable for all users to see the entire logical model (i.e., all the actual relations stored in the database.)
- Consider a person who needs to know a customer's loan number but has no need to see the loan amount. This person should see a relation described, in the relational algebra, by

$$\Pi_{customer\text{-}name,\ loan\text{-}number}\ (borrower \bowtie loan)$$

- Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is called a **view**.

View Definition

- A view is defined using the **create view** statement which has the form

  create view *v* **as** <query expression

  where <query expression> is any legal relational algebra query expression. The view name is represented by *v.*
- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.
- View definition is not the same as creating a new relation by evaluating the query expression
  - Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

View Examples

- Consider the view (named *all-customer*) consisting of branches and their customers.

create view *all-customer* **as**

$$\Pi_{branch\text{-}name,\ customer\text{-}name}\ (depositor \bowtie account)$$
$$\cup\ \Pi_{branch\text{-}name,\ customer\text{-}name}\ (borrower \bowtie loan)$$

- We can find all customers of the Perryridge branch by writing:

$$\Pi_{customer\text{-}name}\ (\sigma_{branch\text{-}name = \text{“Perryridge”}}\ (all\text{-}customer))$$

Updates through View

- Database modifications expressed as views must be translated to modifications of the actual relations in the database.
- Consider the person who needs to see all loan data in the *loan* relation except *amount.* The view given to the person, *branch-loan,* is defined as:

create view *branch-loan* **as**

$$\Pi_{\textit{branch-name, loan-number}} \textit{(loan)}$$

- Since we allow a view name to appear wherever a relation name is allowed, the person may write:

$$\textit{branch-loan} \leftarrow \textit{branch-loan} \cup \{(\text{“Perryridge”, L-37})\}$$

Updates through Views (Contd.)

- The previous insertion must be represented by an insertion into the actual relation *loan* from which the view *branch-loan* is constructed.
- An insertion into *loan* requires a value for *amount*. The insertion can be dealt with by either.
  - rejecting the insertion and returning an error message to the user.
  - inserting a tuple (“L-37”, “Perryridge”, *null*) into the *loan* relation
- Some updates through views are impossible to translate into database relation updates
  - create view v as $\sigma_{\textit{branch-name} = \text{“Perryridge”}}$ (*account*))
    v $\leftarrow$ v $\cup$ (L-99, Downtown, 23)
- Others cannot be translated uniquely
  - *all-customer* $\leftarrow$ *all-customer* $\cup$ {(“Perryridge”, “John”)}
    - Have to choose loan or account, and create a new loan/account number!

Views defined using other views

- One view may be used in the expression defining another view
- A view relation v_1 is said to *depend directly* on a view relation v_2 if v_2 is used in the expression defining v_1
- A view relation v_1 is said to *depend on* view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- A view relation v is said to be *recursive* if it depends on itself.

View Expansion

- A way to define the meaning of views defined in terms of other views.
- Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations.
- View expansion of an expression repeats the following replacement step:

  repeat
  Find any view relation v_i in e_1
  Replace the view relation v_i by the expression defining v_i
  until no more view relations are present in e_1

- As long as the view definitions are not recursive, this loop will terminate

TUPLE RELATIONAL CALCULUS

- A nonprocedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- It is the set of all tuples t such that predicate P is true for t
- t is a *tuple variable*, $t[A]$ denotes the value of tuple t on attribute A
- $t \in r$ denotes that tuple t is in relation r
- P is a *formula* similar to that of the predicate calculus

Predicate Calculus Formula

1. Set of attributes and constants
2. Set of comparison operators: (e.g., $<$, $\leq$, $=$, $\neq$, $>$, $\geq$)
3. Set of connectives: and ($\wedge$), or (v), not ($\neg$)
4. Implication ($\Rightarrow$): x $\Rightarrow$ y, if x if true, then y is true

$$x \Rightarrow y \equiv \neg x \vee y$$

5. Set of quantifiers:
   - $\exists\, t \in r\ (Q(t)) \equiv$ "there exists" a tuple in *t* in relation *r* such that predicate *Q(t)* is true
   - $\forall t \in r\ (Q(t)) \equiv$ *Q* is true "for all" tuples *t* in relation *r*

Banking Example

- *branch (branch-name, branch-city, assets)*
- *customer (customer-name, customer-street, customer-city)*
- *account (account-number, branch-name, balance)*
- *loan (loan-number, branch-name, amount)*
- *depositor (customer-name, account-number)*
- *borrower (customer-name, loan-number)*

EXAMPLE QUERIES

- Find the *loan-number, branch-name,* and *amount* for loans of over $1200

$$\{t \mid t \in loan \wedge t\,[amount] > 1200\}$$

■Find the loan number for each loan of an amount greater than $1200

$$\{t \mid \exists\, s \in \text{loan}\ (t[loan\text{-}number] = s[loan\text{-}number] \wedge s\,[amount] > 1200)\}$$

Notice that a relation on schema [loan-number] is implicitly defined by the query

EXAMPLE QUERIES

- Find the names of all customers having a loan, an account, or both at the bank

$$\{t \mid \exists s \in borrower(\, t[customer\text{-}name] = s[customer\text{-}name]) \\ \vee \exists u \in depositor(\, t[customer\text{-}name] = u[customer\text{-}name])$$

- Find the names of all customers who have a loan and an account at the bank

$$\{t \mid \exists s \in borrower(\, t[customer\text{-}name] = s[customer\text{-}name]) \\ \wedge \exists u \in depositor(\, t[customer\text{-}name] = u[customer\text{-}name])$$

EXAMPLE QUERIES

- Find the names of all customers having a loan at the Perryridge branch

$$\{t \mid \exists s \in \textit{borrower}(t[\textit{customer-name}] = s[\textit{customer-name}] \wedge \exists u \in \textit{loan}(u[\textit{branch-name}] = \text{“Perryridge”} \wedge u[\textit{loan-number}] = s[\textit{loan-number}]))\}$$

- Find the names of all customers who have a loan at the Perryridge branch, but no account at any branch of the bank

$$\{t \mid \exists s \in \textit{borrower}(t[\textit{customer-name}] = s[\text{customer-name}] \wedge \exists u \in \textit{loan}(u[\textit{branch-name}] = \text{“Perryridge”} \wedge u[\textit{loan-number}] = s[\text{loan-}\textit{number}])) \wedge \textbf{not } \exists v \in \textit{depositor}\ (v[\textit{customer-name}] = t[\text{customer-name}])\}$$

Example Queries

- Find the names of all customers having a loan from the Perryridge branch, and the cities they live in

$$\{t \mid \exists s \in loan(s[branch\text{-}name] = \text{“Perryridge”} \wedge \exists u \in borrower\,(u[loan\text{-}number] = s[loan\text{-}number] \wedge t[customer\text{-}name] = u[customer\text{-}name]) \wedge \exists v \in customer\,(u[customer\text{-}name] = v[customer\text{-}name] \wedge t[customer\text{-}city] = v[customer\text{-}city]))\}$$

EXAMPLE QUERIES

- Find the names of all customers who have an account at all branches located in Brooklyn:

$$\{t \mid \exists\, c \in \text{customer}\ (t[\text{customer.name}] = c[\text{customer-name}]) \wedge$$
$$\forall\, s \in \textit{branch}(s[\textit{branch-city}] = \text{“Brooklyn”} \Rightarrow$$
$$\exists\, u \in \textit{account}\ (\ s[\textit{branch-name}] = u[\text{branch-name}]$$
$$\wedge \exists\, s \in \textit{depositor}\ (\ \ t[\textit{customer-name}] = s[\text{customer-name}]$$
$$\wedge\ \ s[\textit{account-number}] = u[\text{account-number}]\ ))\ )\}$$

Safety of Expressions

- It is possible to write tuple calculus expressions that generate infinite relations.
- For example, {t | $\neg$ *t* $\in$ *r*} results in an infinite relation if the domain of any attribute of relation *r* is infinite
- To guard against the problem, we restrict the set of allowable expressions to safe expressions.
- An expression {*t* | *P*(*t*)} in the tuple relational calculus is *safe* if every component of *t* appears in one of the relations, tuples, or constants that appear in *P*
  - NOTE: this is more than just a syntax condition.
    - E.g. { *t* | *t*[*A*]=5 $\vee$ **true** } is not safe --- it defines an infinite set with attribute values that do not appear in any relation or tuples or constants in *P*.

Domain Relational Calculus

- A nonprocedural query language equivalent in power to the tuple relational calculus
- Each query is an expression of the form:

$$\{ < x_1, x_2, \ldots, x_n > \mid P(x_1, x_2, \ldots, x_n)\}$$

  - $x_1, x_2, \ldots, x_n$ represent domain variables
  - P represents a formula similar to that of the predicate calculus

EXAMPLE QUERIES

- Find the *loan-number, branch-name,* and *amount* for loans of over $1200

$$\{< l, b, a > \mid < l, b, a > \in loan \wedge a > 1200\}$$

- Find the names of all customers who have a loan of over $1200

$$\{< c > \mid \exists\, l, b, a\, (< c, l > \in borrower \wedge < l, b, a > \in loan \wedge a > 1200)\}$$

- Find the names of all customers who have a loan from the Perryridge branch and the loan amount:

$$\{< c, a > \mid \exists\, l\, (< c, l > \in borrower \wedge \exists b(< l, b, a > \in loan \wedge b = \text{“Perryridge”}))\}$$

or $\{< c, a > \mid \exists\, l\, (< c, l > \in borrower \wedge < l, \text{“Perryridge”}, a > \in loan)\}$

EXAMPLE QUERIES

- Find the names of all customers having a loan, an account, or both at the Perryridge branch:

$$\{< c > \mid \exists\, l\, (\{< c, l > \in borrower \wedge \exists\, b,a(< l, b, a > \in loan \wedge b = \text{“Perryridge”})) \vee \exists\, a(< c, a > \in depositor \wedge \exists\, b,n(< a, b, n > \in account \wedge b = \text{“Perryridge”}))\}$$

- Find the names of all customers who have an account at all branches located in Brooklyn:

$$\{< c > \mid \exists\, s, n\, (< c, s, n > \in \text{customer}) \wedge \forall\, x,y,z(< x, y, z > \in branch \wedge y = \text{“Brooklyn”}) \Rightarrow \exists\, a,b(< x, y, z > \in account \wedge < c,a > \in depositor)\}$$

SAFETY OF EXPRESSIONS

$$\{ < x_1, x_2, \ldots, x_n > \mid P(x_1, x_2, \ldots, x_n)\}$$

is safe if all of the following hold:

1. All values that appear in tuples of the expression are values from *dom*(*P*) (that is, the values appear either in *P* or in a tuple of a relation mentioned in *P*).
2. For every "there exists" subformula of the form $\exists x (P_1(x))$, the subformula is true if and only if there is a value of *x* in *dom*(P_1) such that $P_1(x)$ is true.
3. For every "for all" subformula of the form $\forall_x (P_1(x))$, the subformula is true if and only if $P_1(x)$ is true for all values *x* from *dom* (P_1).

END
OF
MODULE 3